Calcul de requêtes dans une base de données

Nathan TILLIER

Sommaire

1. Introduction
2. Aspect théorique
3. Algorithmes
4. Implémentation finale
5. Résultats expérimentaux
6. Conclusion

Introduction

Base de données

On va s'intéresser au schéma suivant :

```
CREATE TABLE communes (
    INT id PRIMARY KEY,
    INT region,
    INT departement,
    CHAR nom
)

CREATE TABLE departements (
    INT id PRIMARY KEY,
    INT region,
    CHAR nom
)

CREATE TABLE regions (
    INT id PRIMARY KEY,
    INT cheflieu,
    CHAR nom
)
```

Base de données

Communes

Identifiant	Région	Département	Nom
75056	11	75	Paris
29019	53	29	Brest
69123	84	69	Lyon

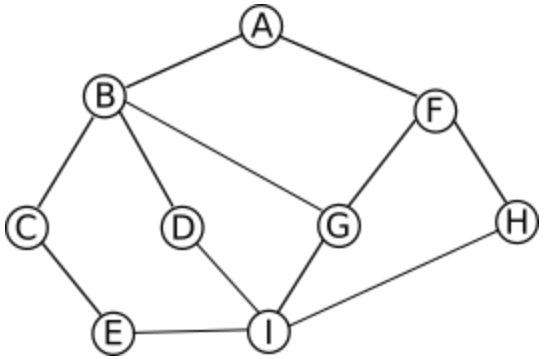
Régions et départements

Identifiant	Cheflieu	Nom
11	75056	Île-de-France
53	35238	Bretagne
84	69123	Auvergne-Rhône-Alpes

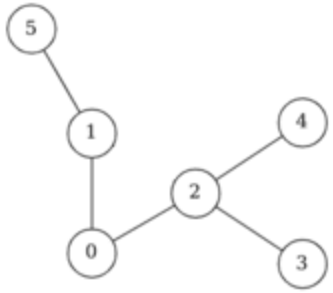
Identifiant	Région	Nom
75	11	Paris
29	53	Finistère
69	84	Rhône

Graphes et acyclicité

Graphe cyclique



Graphe acyclique



Liste d'adjacence

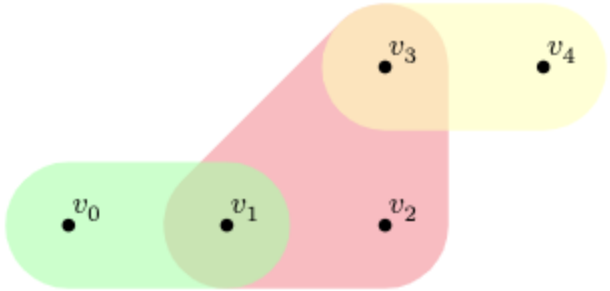
```
A : B, F
B : A, C, D, G
C : B, E
D : B, I
...
```

Liste d'adjacence

```
5 : 1
1 : 0, 5
0 : 1, 2
2 : 0, 3, 4
...
```

Hypergraphes

Hypergraphe



Liste d'adjacence

```
Vert : 0, 1
Rouge : 1, 2, 3
Jaune : 4
```

Définition

Soit $X = \{v_0, v_1, \ldots, v_n\}$ un ensemble fini.
Un hypergraphe sur X est une famille
$H = (E_0, E_2, \ldots, E_p)$ de parties de X vérifiant

$$E_i \neq \emptyset \text{ pour } i = 0, 1, \ldots, p$$

Et

$$\bigcup_{i=0}^{p} E_i = X$$

Nathan TILLIER
© Calcul de requêtes dans une base de données

Structure d'hypergraphe

On se munit de fonctions auxiliaires pour manipuler les hypergraphes.

```
class Arete:
    cardinal: int
    field: int
    indice: int

class Hyper:
    aretes: List[Arete]
    ordre: int
```

```
class Arete:
    def sommets(self)
    def copier(self)
    def augmenter(self, sommet: int)
    def reduire(self, sommet: int)
    def dans(self, sommet: int)
    def contient(self, arete: Arete)

class Hyper:
    def nb_aretes(self)
    def decompte(self, sommet: int)
    def est_valide(self)
    def retirer_sommet(self, sommet: int)
    def copier(self)
    def ajouter_arere(self, arete: Arete)
    def enlever_arete(self, arete: int)
```

Aspect théorique

Algèbre relationnelle

$$\text{rel}(a_1 : dom_1, \ldots, a_n : dom_n)$$

Exemple

$$\text{commune}(\text{identifiant} : \text{entier},\ \text{departement} : \text{entier},\ \text{region} : \text{entier},\ \text{nom} : \text{texte})$$

n-uplets

- P = commune(75056, 11, 75, Paris)
- B = commune(29019, 53, 29, Brest)
- L = commune(69123, 84, 69, Lyon)

$$I = \{P, B, L\}$$

Requête conjonctive

$$q(x_1, \ldots, x_n) : -R_1(\vec{y_1}), \ldots, R_p(\vec{y_p})$$

Exemple

- Schéma

$$\text{commune}(\text{nom} : \text{texte}, \text{departement} : \text{entier})$$

$$\text{departement}(\text{numero} : \text{entier}, \ \text{nom} : \text{texte}, \ \text{region} : \text{texte})$$

- Requête

$$q_1(c, d, r) : -\text{commune}(c, x), \text{departement}(x, d, r)$$

Hypergraphe d'une requête

Définition

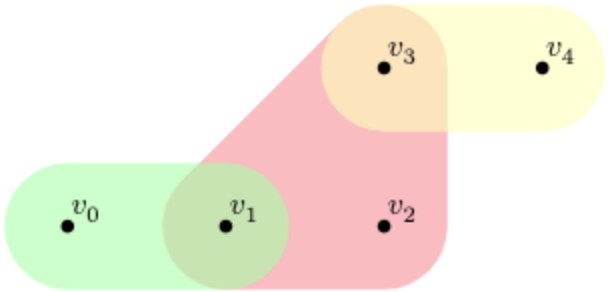
$$q(x_1, \ldots, x_n) :- R_1(\vec{y_1}), \ldots, R_p(\vec{y_p})$$

- $V = \{x_1, \ldots, x_n\}$
- les arêtes sont les variables apparaissant dans chacune des clauses atomiques

$$E_i = V \cap \mathrm{var}(\vec{y_i})$$

Exemple

$$q(v_0, v_1, v_2, v_3, v_4) :- R_{\mathrm{vert}}(v_0, v_1), R_{\mathrm{rouge}}(v_1, v_2, v_3), R_{\mathrm{jaune}}(v_3, v_4)$$



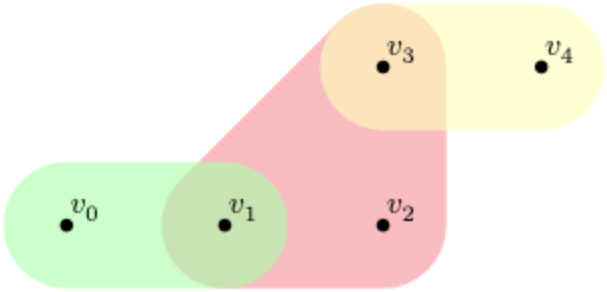
Arbres de jointure

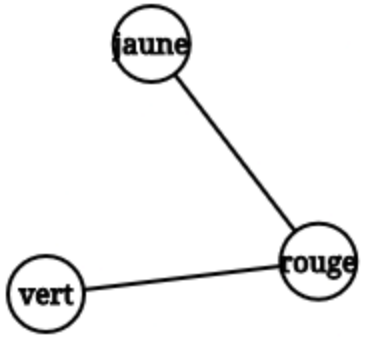
Définition

Soit H un hypergraphe sur X, et T un graphe ayant pour sommet les arêtes de H.
T est un arbre de jointure si

- pour tout sommet v de H, le sous-graphe de T engendré par l'ensemble des sommets de T contenant v est connexe
- T est un arbre

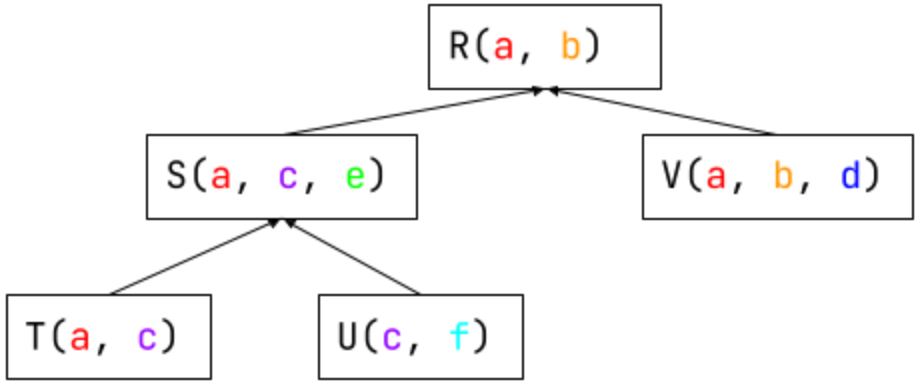
Un tel hypergraphe est dit α-acyclique.





Arbres de jointure

```
q(a, b, c, d, e, f) :- R(a, b), S(a, c, e), T(a, c), U(c, f), V(a, b, d)
```



Algorithmes

Réduction Graham-Yu-Ozsoyoglu

Pour déterminer si un hypergraphe est α-acyclique.

Entrée : hypergraphe H
Sortie : booléen indiquant si H est acyclique

- **Tant que** H a une oreille **faire**
  - e oreille de H
  - retirer tous les sommets exclusivement dans e
  - retirer e
- **Fin**
- **Renvoyer** H est l'hypergraphe vide

```python
def est_acyclique(self):
    h = self.copier()
    restant = set(range(h.ordre))
    sommets_compte = [h.decompte(s) for s in range(h.ordre)]

    while True:
        sortir = True

        # On enlève tous les sommets isolés
        for s in restant:
            if sommets_compte[s] <= 1:
                restant.discard(s)
                h.retirer_sommet(s)
                sortir = False

        # On enlève toutes les arêtes inclues dans d'autre
        for indice, a in enumerate(h.aretes):
            for b in h.aretes:
                if a.dans(b) and a.indice != b.indice:
                    for s in a.sommets:
                        sommets_compte[s] -= 1
                    h.enlever_arete(indice)
                    sortir = False
                    break

        # On continue éventuellement l'algorithme
        if sortir or len(restant) == 0:
            break

    return len(restant) == 0
```

Construction de l'arbre de jointure

Il suffit de reprendre l'algorithme précédent, en rajoutant des arêtes dans l'arbre au fur et à mesure que des sommets de l'hypergraphe sont retirés.

```
def est_acyclique(self):
    arbre = [[] for _ in range(self.nb_aretes)]

    # [...]

    while True:
        # [...]

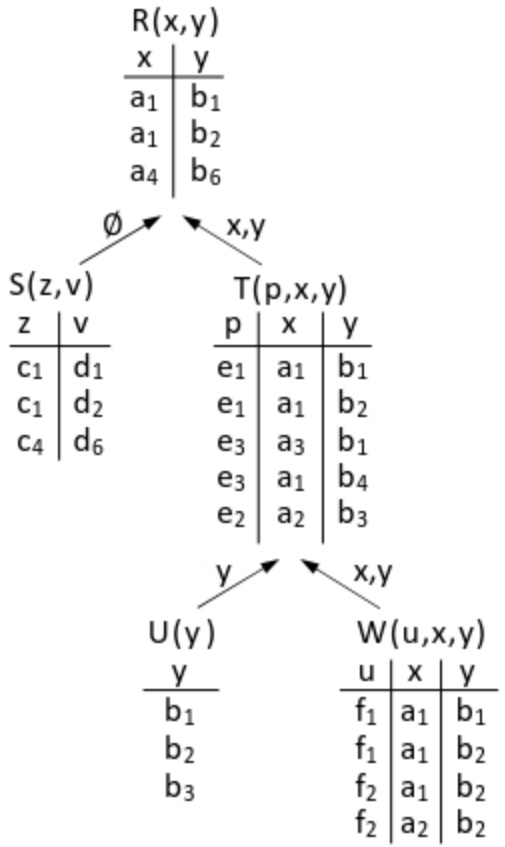
        # On enlève toutes les arêtes inclues
        for indice, a in enumerate(h.aretes):
            for b in h.aretes:
                if a.dans(b) and a.indice != b.indice:
                    arbre[a.indice].append(b.indice)
                    arbre[b.indice].append(a.indice)
                    # [...]

        # [...]

    return arbre if len(restant) == 0 else None
```

Algorithme de Yannakakis

- **Entrée :** une requête conjonctive q, une instance I
- **Sortie :** $q(I)$
- $T \leftarrow$ arbre de jointure de q avec une racine
- $J \leftarrow$ instance réduite issue de I
- $R_1, \ldots, R_n \leftarrow$ parcours postfixe de T
- **Pour** $i = 1, \ldots, n$ **faire**
  - $S_1, \ldots, S_k$ enfants de R_i dans T
  - **Pour** $j = 1, \ldots, k$ **faire**
    - $R_i^J \leftarrow \pi_{\text{vars}(R_i) \cup \text{head}(q)}(R_i^J \bowtie S_j^J)$
  - **Fin**
- **Fin**



Algorithme de Yannakakis

```
# Semi-jointure
def semi_join(self, node: Self):
    intersection = list(self.variables.intersection(node.variables))

    index = set([
        tuple(row[node.record[v]] for v in intersection) for row in node.table.rows
    ])

    self.table.project(
        [v in self.variables for v in self.table.columns],
        lambda row: tuple(row[self.record[v]] for v in intersection in index
    )

    for i, v in enumerate(self.table.columns):
        self.record[v] = i

# Passage bottom -> up
def bottom_up(self, query: Query, node: JoinNode):
    for child in node.children:
        self.bottom_up(query, child)

        variables = node.variables.intersection(child.variables)
        if len(variables) > 0:
            child.semi_join(node)
```

Implémentation

Requête

Construction d'une requête

```
nom_com = "nom_com"
nom_dep = "nom_dep"
nom_reg = "nom_reg"
num_dep = "num_dep"
num_reg = "num_reg"

_ = "_"

r = Query(
    Head("requête", nom_com, nom_dep, nom_reg),
    Atom("commune", _, _, num_reg, num_dep, _, _, _, _, _, nom_com, _, _),
    Atom("département", num_dep, _, _, _, _, _, nom_dep),
    Atom("région", num_reg, _, _, _, _, nom_reg),
)
```

Construction de la base de données

```
class Table:
    name: str
    rows: List[tuple]
    columns: List[str]

    def len(self)
    def import_from(self, csv_path: str)
    def export_to(self, csv_path: str)
    def project(self, selection: List[bool], validate)
```

```
class Instance:
    tables: dict[str, Table]

    def add_table(self, table: Table)
    def jointure(self, query: Query)

    def bottom_up(self, query: Query, node: JoinNode)
    def top_down(self, query: Query, node: JoinNode)
    def query(self, Query)
```

```
i = Instance()

communes = Table("commune")
communes.import_from("code/data/v_commune_2025.csv")

i.add_table(communes)
```

Réalisation d'une requête

```
# Réalisation d'une requête
def query(self, query: Query):

    # Vérification de la requête
    for atom in query.atoms:
        if atom.name not in self.tables:
            raise InterruptedError(atom.name + " non présent dans les tables.")
        if len(atom.original) != len(self.tables[atom.name].columns):
            raise InterruptedError("Nombre de colonnes non correspondant dans la table " + atom.name)

    # Création de l'arbre de jointure
    jointree = self.jointure(query)

    # Premier passage
    self.bottom_up(query, jointree)

    # Deuxième passage
    self.top_down(query, jointree)

    # On réalise la jointure
    jointree.compute(query.head)

    return jointree.table
```

Réalisation d'une requête

```
def jointure(self, query: Query):
    hyper = Hyper(len(query.variables))
    record = dict()

    for i, v in enumerate(query.variables):
        record[v] = i

    for atom in query.atoms:
        arete = Arete()
        arete.augmenter(*[record[v] for v in atom.variables])
        hyper.ajouter_arete(arete)

    arbre = hyper.arbre_jointure()

    if not arbre:
        raise InterruptedError("Requête non cyclique.")

    vus = set()
    return self.join_dfs(arbre, query, vus, 0)

def compute(self, head: Head):
    for index, child in enumerate(self.children):
        child.compute(head)
        self.full_join(child, head, self.root and index == len(self.children) - 1)
```

Réalisation d'une requête

```
# Jointure complète
def full_join(self, node: Self, head: Head, strict):
    intersection = list(self.variables.intersection(node.variables))
    index  = defaultdict(list)

    # Construction de la sélection
    selection = []
    columns = []
    for v in head.variables if strict else self.variables.union(node.variables):
        if v in self.variables or v in node.variables:
            current = v in self.variables
            columns.append(v)
            selection.append(
                (current, self.record[v] if current else node.record[v])
            )
        else:
            raise InterruptedError(v + " non présent lors de la jointure.")

    # Construction du résultat
    for row in node.table.rows:
        key = tuple(row[node.record[v]] for v in intersection)
        index[key].append(row)

    result = []
    for row in self.table.rows:
        key = tuple(row[self.record[v]] for v in intersection)

        if key in index:
            for row2 in index[key]:
                result.append(self.patch_rows(row, row2, selection))

    self.table.rows = result
    self.table.columns = columns
    self.record = { v:i for i, v in enumerate(columns) }
    self.variables = set(head.variables) if strict else self.variables.union(node.variables)
```

Résultats expérimentaux

Exemple de requête

Commune, département, région

```
SELECT commune.nom, departement.nom, region.nom FROM commune
JOIN departement ON departement.numero = commune.departement
JOIN region ON region.numero = commune.region
```

```
Query(
    Head("requête", nom_com, nom_dep, nom_reg),
    Atom("commune", _, _, num_reg, num_dep, _, _, _, _, _, nom_com, _, _),
    Atom("département", num_dep, _, _, _, _, _, nom_dep),
    Atom("région", num_reg, _, _, _, _, nom_reg),
)
```

Résultats :

- Version simple : 0.17 secondes
- Version double parcours : 0.08 secondes

Exemple de requête

Commune, cheflieu

```
SELECT commune.nom, departement.nom, cheflieu.nom FROM commune
JOIN departement ON departement.numero = commune.departement
JOIN commune as cheflieu ON commune.numero = departement.cheflieu
```

```
Query(
    Head("requête", nom_com, nom_cheflieu),
    Atom("commune", _, _, _, num_dep, _, _, _, _, _, nom_com, _, _),
    Atom("département", num_dep, _, num_cheflieu, _, _, _, _),
    Atom("commune", _, num_cheflieu, _, _, _, _, _, _, _, nom_cheflieu, _, _)
)
```

Résultats :

- Version simple : 0.09 secondes
- Version double parcours : 0.06 secondes

Conclusion

Conclusion

Concernant l'efficacité

- Le code est très proche de l'algorithme théorique, et il y a de nombreux éléments recalculés.
- Il est possible de ne pas copier les valeurs directement, mais des pointeurs vers les valeurs de la table (indices), pour consommer moins de mémoire.
- On peut le programmer en C pour de meilleurs performances en temps, et une gestion de la mémoire plus délicate.
- Il est possible de paralléliser l'algorithme, avec un fil réalisant chaque fils de la racine.
- La constante est assez élevée, dans la pratique ce n'est pas l'algorithme qui est utilisé.

Concernant les requêtes

- On peut étendre la structure de requête pour ajouter des conditions sur les valeurs lors du premier passage (bottom-up).

Annexe

Réduction GYO

Éléments de preuve (correction)

- dès qu'on retire un sommet isolé, on ne perd aucune information sur T arbre de jointure. On retire d'ailleurs son arête avec si elle ne contient que lui.
- si une arête E est incluse dans une autre arête F, alors les sommets E et F sont nécessairement reliés dans T (on crée la connexité sans briser l'acyclicité).
- si une de ces étapes n'est pas faisable et que l'hypergraphe n'est pas vide, on ne peut pas contruire T : on créera forcément un cycle.

Éléments de preuve (complexité)

- Enlever les sommets isolés : $O(|X|)$
- Enlever les arêtes : $O(|H|^2|X|)$
- Boucle *while*: au plus $|X| + |H|$ itérations (on retire à chaque fois au moins un sommet ou une arête)

Algorithme de Yannakakis

Éléments de preuve (correction)

- À faire

Éléments de preuve (complexité)

- To show that we obtain the desired complexity, we will prove that no intermediate result can grow more than a polynomial in the size of the input and output. Consider some node with atom R and children S1, . . . , Sn.
- Then the algorithm will replace this node with a new node T' = $\pi_{Y \cup Z}(T)$, where T = R ./ S1 ./ . . . Sn, Y are the variables in R, and Z are the variables in head(q) \ Y.
- Since Y, Z are disjoint, $\pi_{Y \cup Z}(T) \subseteq \pi_Y(T) \times \pi_Z(T)$. Now notice that $\pi_Y(T)$ is at most the size of R. Also, $\pi_Z(T)$ is bounded by the size of the output (but only because we took care of dangling tuples by applying the full reducer).

NP-complétude (calcul de requêtes)

Éléments de preuve (dans NP)

- Certificat : valeurs prises par chaque variables de la requête (de l'en-tête et du corps), polynomial en la taille de celle-ci. Homomorphisme.
- Vérification : la relation est-elle dans la base de données ? Linéaire en la taille de la base de données.

NP-complétude (calcul de requêtes)

Éléments de preuve (NP-difficile)

- On construit une instance et une requête permettant de réduire le problème choisi.
- 3-SAT :
  - L'instance contient toutes les clauses positives (un terme à 1), et les négations pour les variables temporaires : $\neg x_i \equiv x'_i$ et $x_i \vee x'_i$
  - $I = \{Clause(0,0,1), \ldots, Clause(1,1,1)\} \cup \{Negation(0,1), Negation(1,0)\}$
  - Requête : $q(x_1, \ldots, x_k) : -\{Clause(x_i, x_j, x_k)\}, \{Negation(x_i, x'_i)\}$
- k-clique : avec $G = (S, A)$
  - Instance : $I = \{Noeud(s) | s \in S\} \cup \{Arete(s,t); \{s,t\} \in A\}$
  - Requête :
    $q(x_1, \ldots, x_k) : -Noeud(x_1), \ldots, Noeud(x_k), \{Arete(x_i, x_j); i \neq j \in [1,k]\}$